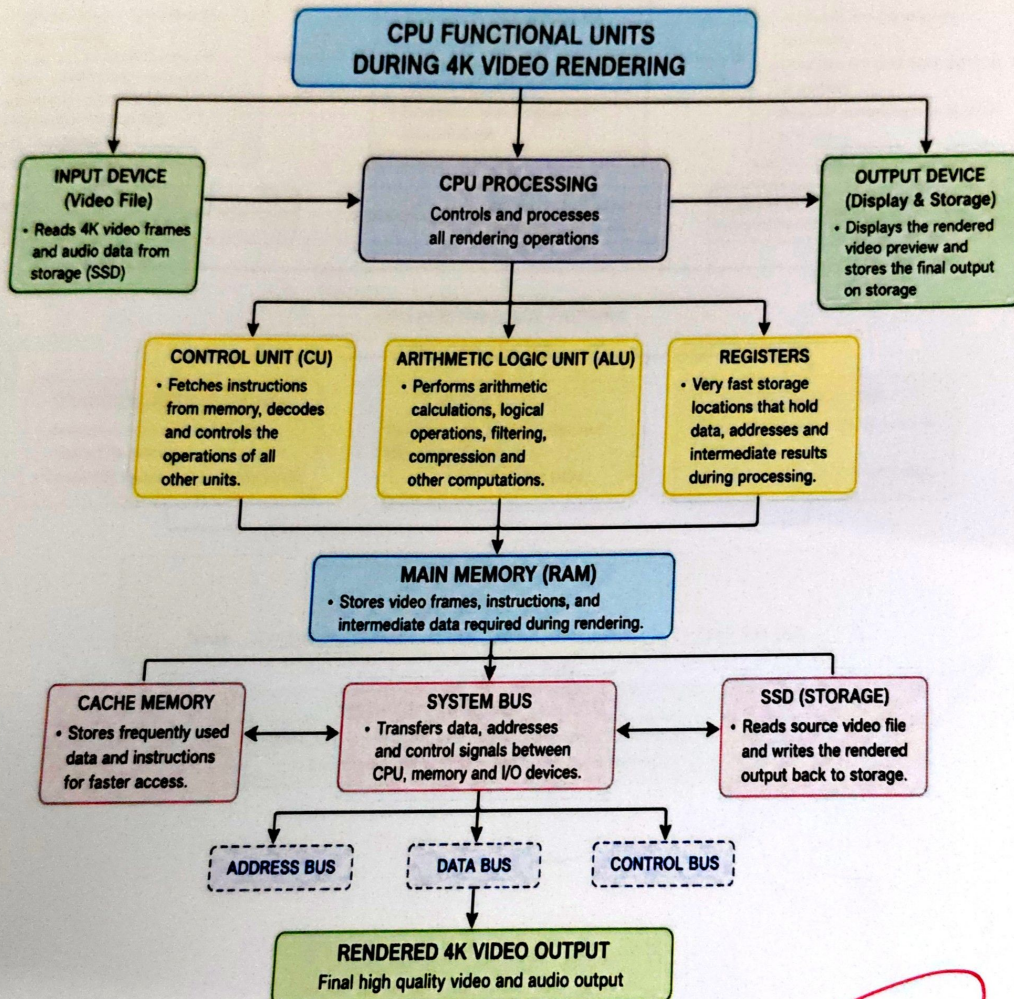


CSA 1220 COMPUTER ARCHITECTURE

CO 1 AT-2 SET-1

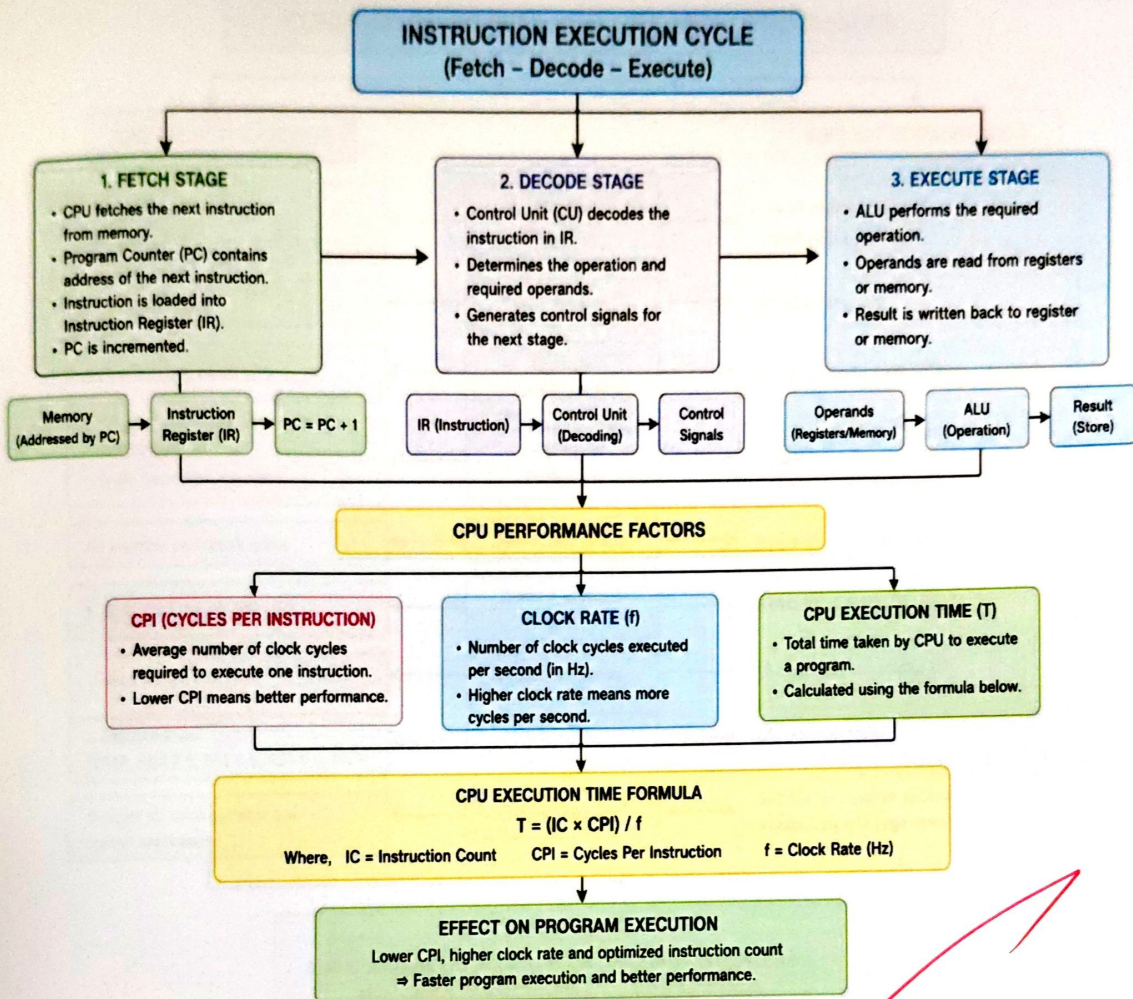
QUESTION 1

CPU Functional Units During 4K Video Rendering



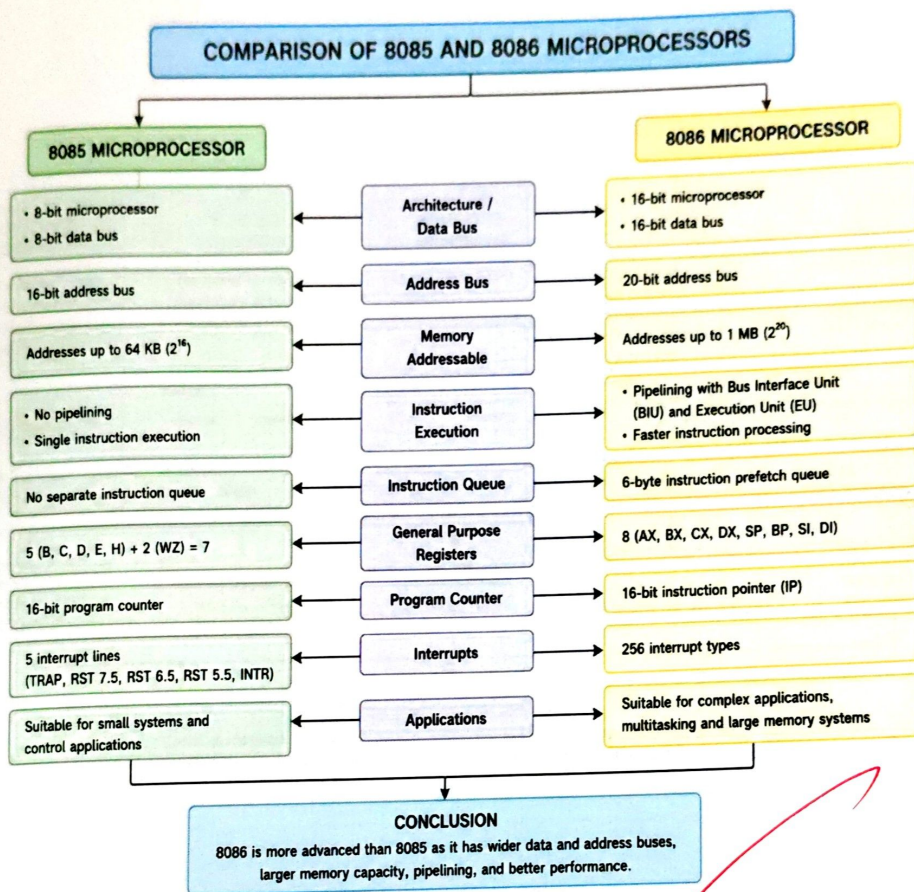
QUESTION 2

Instruction Execution Cycle



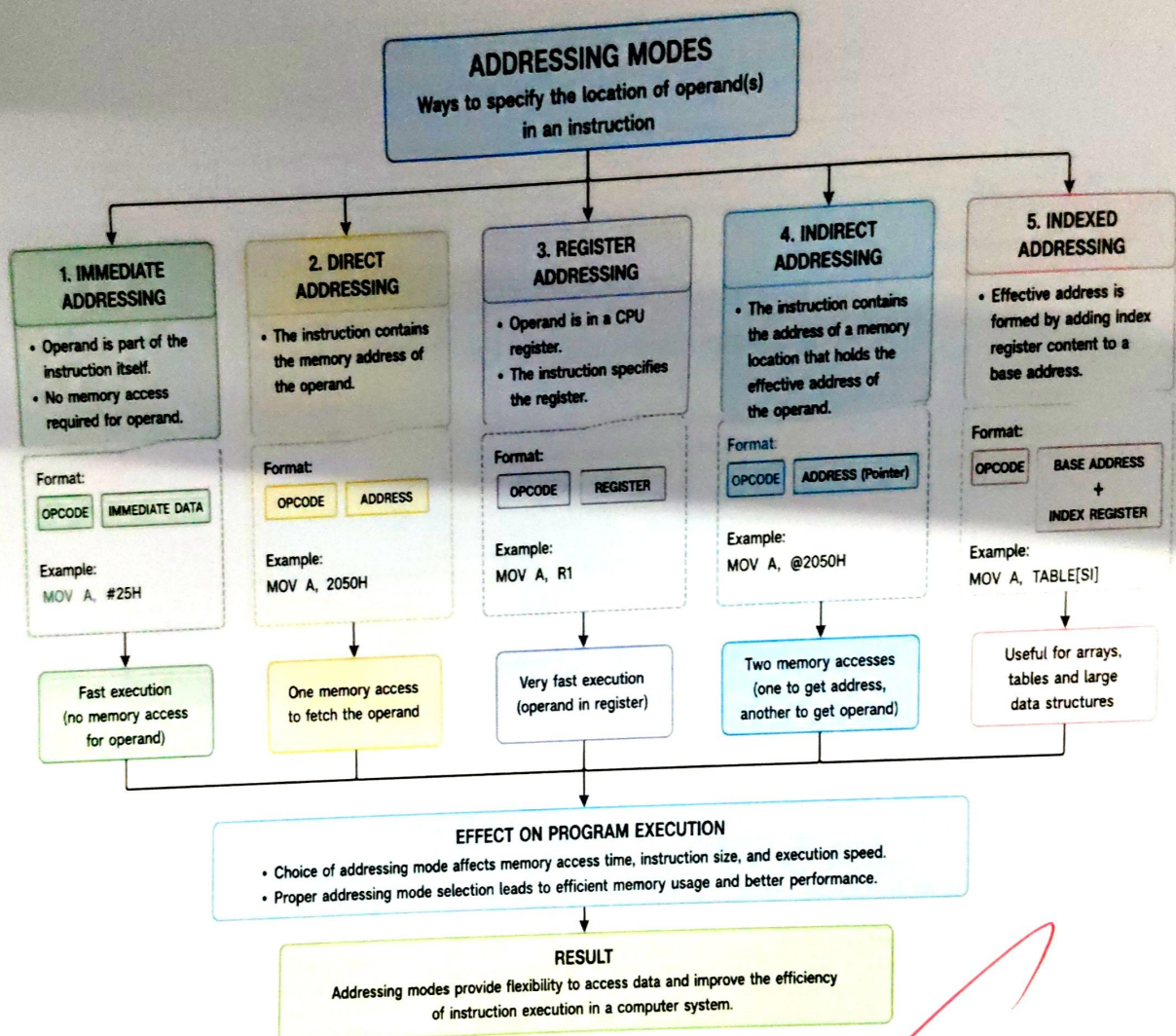
QUESTION 3

Comparison of 8085 and 8086



QUESTION 4

Addressing Modes



QUESTION 5

Binary and Hexadecimal Number Systems

BINARY & HEXADECIMAL NUMBER SYSTEM, MEMORY ORGANIZATION, CPU EXECUTION & DEBUGGING

1. BINARY NUMBER SYSTEM

- Uses only two digits: 0 and 1.
- Base (radix) = 2
- Used internally by CPU for all data and instructions.
- Example: $(101101)_2$

2. HEXADECIMAL NUMBER SYSTEM

- Uses 16 symbols: 0-9 and A-F.
- Base (radix) = 16
- Compact representation of binary data.
- Each hex digit = 4 bits.
- Example: $(2AF3)_{16}$

3. RELATIONSHIP & CONVERSION

Hex Digit	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Binary (4 bits)	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

- Binary $\leftrightarrow$ Hexadecimal conversion simplifies data handling.
- Easy to read, write and debug using hexadecimal.

4. MEMORY ORGANIZATION

- Memory is organized as a sequence of locations (addresses).
- Each location has a unique address.
- Stores: Instructions, Data, and Results.
- Types:
 - ROM
 - RAM
 - Cache
 - Registers
- Address bus selects location, data bus transfers data.

5. CPU EXECUTION CYCLE

1. FETCH

- CPU fetches instruction from memory using Program Counter (PC).
- Instruction is loaded into Instruction Register (IR).

2. DECODE

- Control Unit decodes the instruction.
- Determines operation and required operands.

3. EXECUTE

- ALU performs the operation.
- Data is read from registers/memory.

4. STORE / WRITE BACK

- Result is written back to register or memory.
- Flags are updated.
- PC is updated for next instruction.

Uses Binary & Hex Data

- Instructions and data are in binary.
- Displayed/Debugged in hexadecimal for clarity.
- Execution depends on memory organization and addressing.

6. DEBUGGING PROCESS

1. SET BREAKPOINTS

- Set breakpoints at specific addresses or instructions.

2. RUN / STEP EXECUTION

- Run program or execute step by step.
- Observe register values, memory and flags.

3. MONITOR IN HEX

- View memory contents, registers, and data in hexadecimal.
- Easier to detect patterns and errors.

4. IDENTIFY ISSUES

- Check incorrect values, wrong addresses, logic errors, or invalid instructions.

5. FIX AND VERIFY

- Correct the code or data.
- Re-run and verify expected results.

Tools Used

- Debuggers, Emulators, Assemblers
- Memory Dump, Register View, Breakpoints, Watchpoints

OVERALL RESULT

- Binary is the language of computers, while Hexadecimal simplifies representation and analysis.
- Proper memory organization enables efficient storage and access of instructions and data.
- CPU execution follows a structured cycle to process instructions.
- Debugging with hexadecimal representation helps find and fix errors, ensuring correct program execution.